

Mini-Project 3: Classification, Decision Rules, and Flexibility

EN.553.434/634: Elements of Statistical Learning

Jonathan Ma

Setting things up

```
set.seed(434634)

library(ISLR2)
library(MASS)

data("OJ")

# Recode response: MM = 1, CH = 0
OJ$Y <- ifelse(OJ$Purchase == "MM", 1, 0)

# Mean squared classification utilities
classification_metrics <- function(y_true, prob, threshold = 0.5) {
  y_hat <- ifelse(prob >= threshold, 1, 0)

  TP <- sum(y_true == 1 & y_hat == 1)
  TN <- sum(y_true == 0 & y_hat == 0)
  FP <- sum(y_true == 0 & y_hat == 1)
  FN <- sum(y_true == 1 & y_hat == 0)

  TPR <- TP / (TP + FN)
  FPR <- FP / (FP + TN)
  error <- mean(y_hat != y_true)

  c(
    TPR = TPR,
    FPR = FPR,
    misclassification_rate = error
  )
}

binary_log_loss <- function(y_true, prob) {
  eps <- 1e-15
  prob <- pmin(pmax(prob, eps), 1 - eps)
```

```

    -mean(
      y_true * log(prob) +
      (1 - y_true) * log(1 - prob)
    )
  }

auc_score <- function(y_true, prob) {
  n1 <- sum(y_true == 1)
  n0 <- sum(y_true == 0)

  ranks <- rank(prob, ties.method = "average")
  rank_sum_1 <- sum(ranks[y_true == 1])

  (rank_sum_1 - n1 * (n1 + 1) / 2) / (n1 * n0)
}

roc_points <- function(y_true, prob) {
  thresholds <- sort(
    unique(c(Inf, prob, -Inf)),
    decreasing = TRUE
  )

  out <- lapply(thresholds, function(t) {
    y_hat <- ifelse(prob >= t, 1, 0)

    TP <- sum(y_true == 1 & y_hat == 1)
    TN <- sum(y_true == 0 & y_hat == 0)
    FP <- sum(y_true == 0 & y_hat == 1)
    FN <- sum(y_true == 1 & y_hat == 0)

    data.frame(
      threshold = t,
      TPR = ifelse(TP + FN == 0, NA, TP / (TP + FN)),
      FPR = ifelse(FP + TN == 0, NA, FP / (FP + TN))
    )
  })

  do.call(rbind, out)
}

knn_prob <- function(X_train, y_train, X_new, K) {
  X_train <- as.matrix(X_train)
  X_new <- as.matrix(X_new)

  if (is.null(dim(X_new))) {
    X_new <- matrix(X_new, ncol = ncol(X_train))
  }
}

```

```

sapply(seq_len(nrow(X_new)), function(j) {
  d <- sqrt(
    rowSums(
      (X_train -
        matrix(
          X_new[j, ],
          nrow(X_train),
          ncol(X_train),
          byrow = TRUE
        ))^2
    )
  )
  neighbors <- order(d)[1:K]
  mean(y_train[neighbors])
})
}

simulate_classes <- function(n, mu0, Sigma0, mu1, Sigma1) {
  n0 <- floor(n / 2)
  n1 <- n - n0

  X0 <- MASS::mvrnorm(n0, mu = mu0, Sigma = Sigma0)
  X1 <- MASS::mvrnorm(n1, mu = mu1, Sigma = Sigma1)

  X <- rbind(X0, X1)
  y <- c(rep(0, n0), rep(1, n1))

  data.frame(
    X1 = X[, 1],
    X2 = X[, 2],
    y = y
  )
}

make_grid_2d <- function(X1, X2, n = 200, expand = 0.1) {
  r1 <- range(X1)
  r2 <- range(X2)

  d1 <- diff(r1)
  d2 <- diff(r2)

  x1_grid <- seq(
    r1[1] - expand * d1,
    r1[2] + expand * d1,
    length.out = n
  )
}

```

```

x2_grid <- seq(
  r2[1] - expand * d2,
  r2[2] + expand * d2,
  length.out = n
)

expand.grid(
  X1 = x1_grid,
  X2 = x2_grid
)
}

```

1. Logistic regression and estimated probabilities

Use the OJ dataset. Recode the response as

$$Y = \begin{cases} 1, & \text{MM,} \\ 0, & \text{CH.} \end{cases}$$

Randomly select 800 observations for the training set D and the rest as the test set T . Using D , fit logistic regression with `LoyalCH` as the only predictor.

```

set.seed(434634)

# Train/test split
train_id <- sample(seq_len(nrow(OJ)), 800)

OJ_train <- OJ[train_id, ]
OJ_test  <- OJ[-train_id, ]

# Logistic regression
logit_1 <- glm(
  Y ~ LoyalCH,
  data = OJ_train,
  family = binomial
)

summary(logit_1)

```

Call:

```
glm(formula = Y ~ LoyalCH, family = binomial, data = OJ_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	2.6259	0.2226	11.79	<2e-16 ***

```
LoyalCH      -5.7399      0.3961  -14.49   <2e-16 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 1074.34  on 799  degrees of freedom
Residual deviance:  716.74  on 798  degrees of freedom
AIC: 720.74
```

Number of Fisher Scoring iterations: 5

```
# Grid for fitted probability curve
loyal_grid <- seq(
  min(OJ_train$LoyalCH),
  max(OJ_train$LoyalCH),
  length.out = 400
)

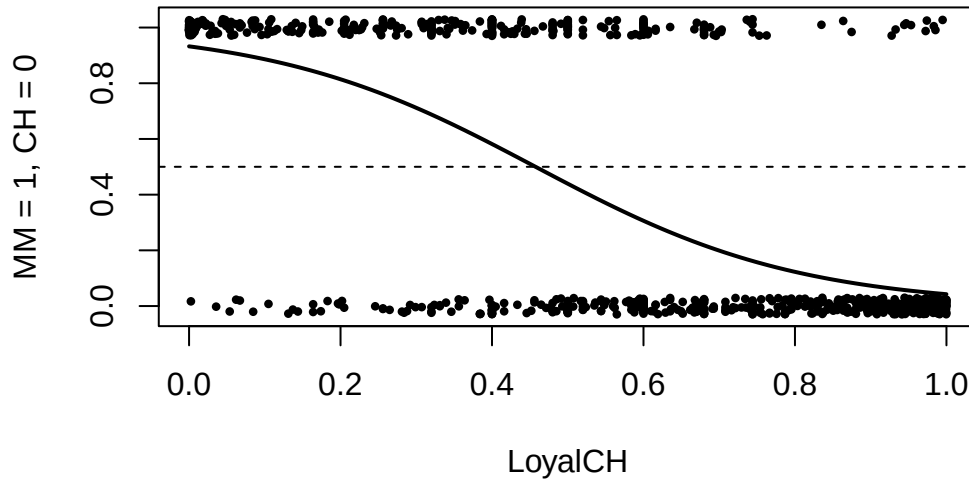
prob_grid <- predict(
  logit_1,
  newdata = data.frame(LoyalCH = loyal_grid),
  type = "response"
)

# Plot observations with slight vertical jitter
plot(
  OJ_train$LoyalCH,
  jitter(OJ_train$Y, amount = 0.03),
  xlab = "LoyalCH",
  ylab = "MM = 1, CH = 0",
  main = "Logistic Regression: Probability of Purchasing MM",
  pch = 16,
  cex = 0.6
)

lines(
  loyal_grid,
  prob_grid,
  lwd = 2
)

abline(
  h = 0.5,
  lty = 2
)
```

Logistic Regression: Probability of Purchasing MM



The $t = 0.5$ decision boundary can be obtained directly from the fitted logistic regression coefficients. Since

$$\text{logit}\{\widehat{P}(Y = 1 \mid X = x)\} = \hat{\beta}_0 + \hat{\beta}_1 x,$$

the probability equals 0.5 when the log-odds equal zero.

```
beta_logit <- coef(logit_1)

decision_boundary <-
  -beta_logit[1] / beta_logit[2]

decision_boundary
```

```
(Intercept)
0.4574784
```

Q1. What quantity is the fitted logistic curve estimating?

The fitted logistic curve estimates the conditional probability

$$\widehat{P}(Y = 1 \mid \text{LoyalCH} = x) = \widehat{P}(\text{Purchase} = \text{MM} \mid \text{LoyalCH} = x).$$

It estimates the probability of purchasing MM for a given value of LoyalCH.

Q2. Using threshold $t = 0.5$, approximately where is the decision boundary?

The estimated decision boundary occurs at approximately $\text{LoyalCH} = 0.457$, where the fitted probability of purchasing MM equals 0.5. Values on the side where the fitted probability exceeds 0.5 are classified as MM, while values on the other side are classified as CH.

2. Classification threshold and ROC

Using the fitted logistic regression model from Part 1, classify the test observations using

$t = 0.25, \quad 0.50, \quad 0.75.$

```
# Test-set probabilities
prob_test_logit <- predict(
  logit_1,
  newdata = OJ_test,
  type = "response"
)

thresholds <- c(0.25, 0.50, 0.75)

threshold_results <- do.call(
  rbind,
  lapply(thresholds, function(t) {
    metrics <- classification_metrics(
      OJ_test$Y,
      prob_test_logit,
      threshold = t
    )

    data.frame(
      Threshold = t,
      TPR = metrics["TPR"],
      FPR = metrics["FPR"],
      Misclassification_Rate =
        metrics["misclassification_rate"]
    )
  })
)

rownames(threshold_results) <- NULL
threshold_results
```

	Threshold	TPR	FPR	Misclassification_Rate
1	0.25	0.94	0.35882353	0.2481481
2	0.50	0.74	0.12352941	0.1740741
3	0.75	0.52	0.02352941	0.1925926

Now construct the full ROC curve.

```
roc_df <- roc_points(
  OJ_test$Y,
  prob_test_logit
)

plot(
  roc_df$FPR,
  roc_df$TPR,
```

```

type = "l",
lwd = 2,
xlab = "False Positive Rate",
ylab = "True Positive Rate",
main = "ROC Curve: Logistic Regression",
xlim = c(0, 1),
ylim = c(0, 1)
)

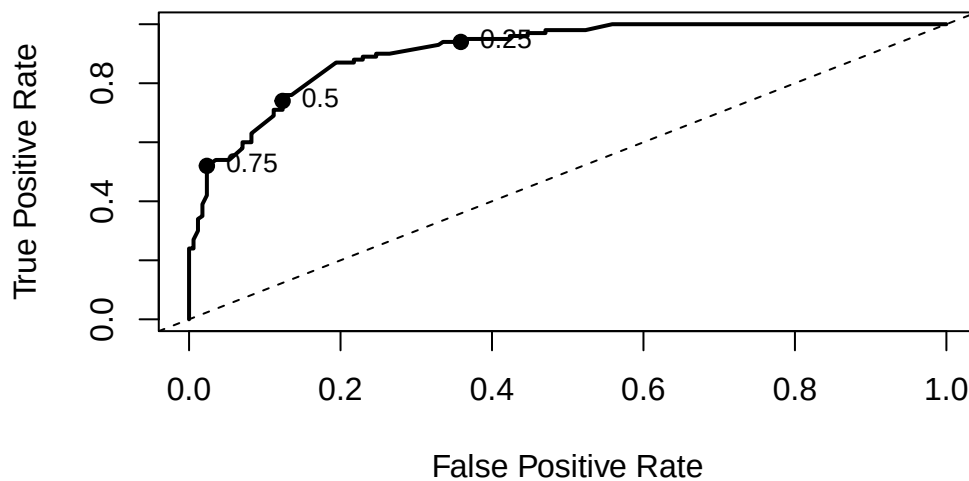
abline(
  0, 1,
  lty = 2
)

# Mark the three requested thresholds
points(
  threshold_results$FPR,
  threshold_results$TPR,
  pch = 19
)

text(
  threshold_results$FPR,
  threshold_results$TPR,
  labels = threshold_results$Threshold,
  pos = 4,
  cex = 0.8
)

```

ROC Curve: Logistic Regression



Q3. What happens to TPR and FPR as the threshold increases?

As the classification threshold increases, fewer observations are classified as positive

(MM). Therefore, both the true positive rate and the false positive rate generally decrease. A higher threshold makes the classifier more conservative about predicting the positive class.

Q4. Does changing the threshold require fitting a new probability model? Explain briefly.

No. The fitted probability model remains unchanged. Changing the threshold only changes how the already estimated probabilities $\hat{P}(Y = 1 \mid X = x)$ are converted into class labels.

3. LDA and QDA: a simulation

Consider a two-class classification problem with equal class probabilities.

```
set.seed(434634)

mu0 <- c(-1, 0)
mu1 <- c(1, 0)

Sigma_equal <- matrix(
  c(1, 0,
    0, 1),
  nrow = 2,
  byrow = TRUE
)

Sigma0 <- matrix(
  c(1, 0,
    0, 1),
  nrow = 2,
  byrow = TRUE
)

Sigma1 <- matrix(
  c(2, 0.8,
    0.8, 0.7),
  nrow = 2,
  byrow = TRUE
)

# Setting A
train_A <- simulate_classes(
  200,
  mu0,
  Sigma_equal,
  mu1,
  Sigma_equal
)
```

```

test_A <- simulate_classes(
  5000,
  mu0,
  Sigma_equal,
  mu1,
  Sigma_equal
)

# Setting B
train_B <- simulate_classes(
  200,
  mu0,
  Sigma0,
  mu1,
  Sigma1
)

test_B <- simulate_classes(
  5000,
  mu0,
  Sigma0,
  mu1,
  Sigma1
)

```

Fit LDA and QDA in Setting A.

```

lda_A <- lda(
  factor(y) ~ X1 + X2,
  data = train_A
)

qda_A <- qda(
  factor(y) ~ X1 + X2,
  data = train_A
)

pred_lda_A <- predict(
  lda_A,
  newdata = test_A
)$class

pred_qda_A <- predict(
  qda_A,
  newdata = test_A
)$class

error_lda_A <- mean(

```

```

    as.numeric(as.character(pred_lda_A)) != test_A$y
  )

error_qda_A <- mean(
  as.numeric(as.character(pred_qda_A)) != test_A$y
)

setting_A_errors <- data.frame(
  Method = c("LDA", "QDA"),
  Test_Error = c(
    error_lda_A,
    error_qda_A
  )
)

setting_A_errors

```

	Method	Test_Error
1	LDA	0.1608
2	QDA	0.1598

Plot the estimated decision boundaries.

```

grid_A <- make_grid_2d(
  train_A$X1,
  train_A$X2
)

lda_grid_A <- predict(
  lda_A,
  newdata = grid_A
)$posterior[, 2]

qda_grid_A <- predict(
  qda_A,
  newdata = grid_A
)$posterior[, 2]

x1_vals_A <- sort(unique(grid_A$X1))
x2_vals_A <- sort(unique(grid_A$X2))

z_lda_A <- matrix(
  lda_grid_A,
  nrow = length(x1_vals_A),
  ncol = length(x2_vals_A)
)

z_qda_A <- matrix(

```

```

qda_grid_A,
nrow = length(x1_vals_A),
ncol = length(x2_vals_A)
)

plot(
  train_A$X1,
  train_A$X2,
  pch = ifelse(train_A$y == 1, 17, 16),
  xlab = "X1",
  ylab = "X2",
  main = "Setting A: LDA and QDA Boundaries"
)

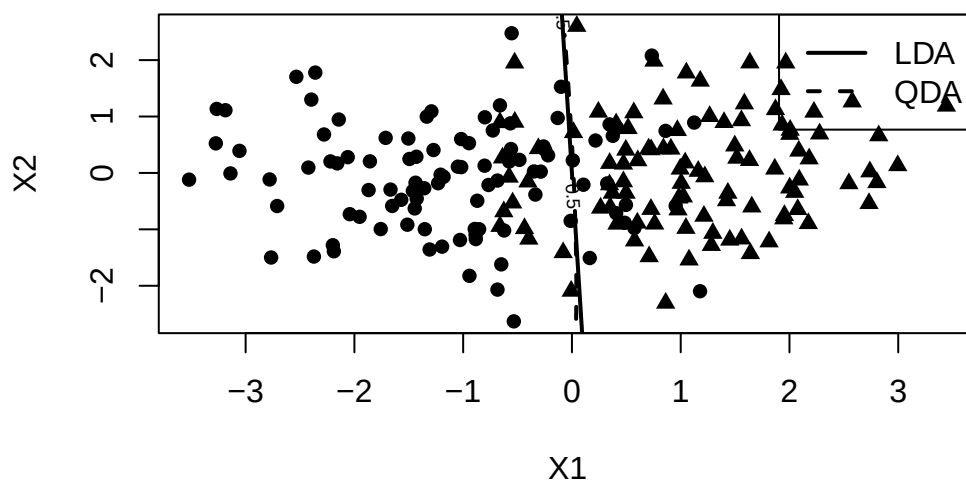
contour(
  x1_vals_A,
  x2_vals_A,
  z_lda_A,
  levels = 0.5,
  add = TRUE,
  lwd = 2
)

contour(
  x1_vals_A,
  x2_vals_A,
  z_qda_A,
  levels = 0.5,
  add = TRUE,
  lwd = 2,
  lty = 2
)

legend(
  "topright",
  legend = c("LDA", "QDA"),
  lwd = 2,
  lty = c(1, 2)
)

```

Setting A: LDA and QDA Boundaries



Now fit LDA and QDA in Setting B.

```
lda_B <- lda(  
  factor(y) ~ X1 + X2,  
  data = train_B  
)  
  
qda_B <- qda(  
  factor(y) ~ X1 + X2,  
  data = train_B  
)  
  
pred_lda_B <- predict(  
  lda_B,  
  newdata = test_B  
)$class  
  
pred_qda_B <- predict(  
  qda_B,  
  newdata = test_B  
)$class  
  
error_lda_B <- mean(  
  as.numeric(as.character(pred_lda_B)) != test_B$y  
)  
  
error_qda_B <- mean(  
  as.numeric(as.character(pred_qda_B)) != test_B$y  
)  
  
setting_B_errors <- data.frame(  
  Method = c("LDA", "QDA"),
```

```

Test_Error = c(
  error_lda_B,
  error_qda_B
)
)

```

```
setting_B_errors
```

```

      Method Test_Error
1      LDA      0.1890
2      QDA      0.1798

```

```

grid_B <- make_grid_2d(
  train_B$X1,
  train_B$X2
)

```

```

lda_grid_B <- predict(
  lda_B,
  newdata = grid_B
)$posterior[, 2]

```

```

qda_grid_B <- predict(
  qda_B,
  newdata = grid_B
)$posterior[, 2]

```

```

x1_vals_B <- sort(unique(grid_B$X1))
x2_vals_B <- sort(unique(grid_B$X2))

```

```

z_lda_B <- matrix(
  lda_grid_B,
  nrow = length(x1_vals_B),
  ncol = length(x2_vals_B)
)

```

```

z_qda_B <- matrix(
  qda_grid_B,
  nrow = length(x1_vals_B),
  ncol = length(x2_vals_B)
)

```

```

plot(
  train_B$X1,
  train_B$X2,
  pch = ifelse(train_B$y == 1, 17, 16),
  xlab = "X1",
  ylab = "X2",

```

```

    main = "Setting B: LDA and QDA Boundaries"
  )

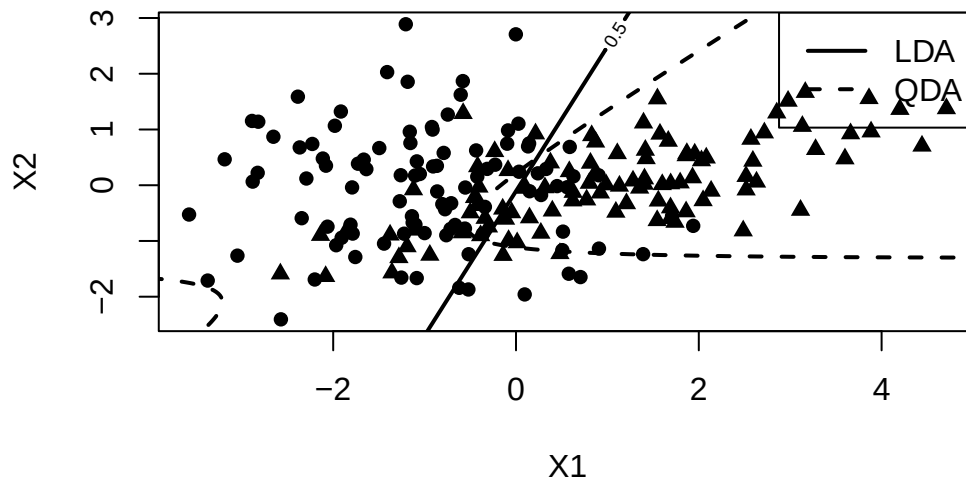
  contour(
    x1_vals_B,
    x2_vals_B,
    z_lda_B,
    levels = 0.5,
    add = TRUE,
    lwd = 2
  )

  contour(
    x1_vals_B,
    x2_vals_B,
    z_qda_B,
    levels = 0.5,
    add = TRUE,
    lwd = 2,
    lty = 2
  )

  legend(
    "topright",
    legend = c("LDA", "QDA"),
    lwd = 2,
    lty = c(1, 2)
  )
)

```

Setting B: LDA and QDA Boundaries



Q5. In Setting A, compare the LDA and QDA decision boundaries and test errors.

In Setting A, the true covariance matrices are equal, so the Bayes decision boundary

is linear and LDA is correctly specified. The estimated LDA and QDA boundaries are similar, and their test errors are nearly identical: 0.1608 for LDA and 0.1598 for QDA. Although QDA happens to have a slightly lower error in this simulation, LDA is better matched to the data-generating process and estimates fewer parameters.

Q6. What changes in Setting B? Which method is better suited to this setting? Do the test errors support your answer?

In Setting B, the two classes have different covariance matrices, so the Bayes decision boundary is generally curved rather than linear. QDA is better suited because it estimates a separate covariance matrix for each class, whereas LDA forces them to be equal. The test errors are 0.189 for LDA and 0.1798 for QDA, so the simulated test results support the expectation that QDA is better suited to Setting B.

4. Flexibility of kNN classification

Return to the OJ data and use `LoyalCH` as the only predictor.

```
K_values <- c(1, 5, 20, 50)

X_train_1d <- matrix(
  OJ_train$LoyalCH,
  ncol = 1
)

loyal_knn_grid <- seq(
  min(OJ_train$LoyalCH),
  max(OJ_train$LoyalCH),
  length.out = 400
)

X_grid_1d <- matrix(
  loyal_knn_grid,
  ncol = 1
)

knn_grid_probs <- sapply(
  K_values,
  function(K) {
    knn_prob(
      X_train_1d,
      OJ_train$Y,
      X_grid_1d,
      K
    )
  }
)

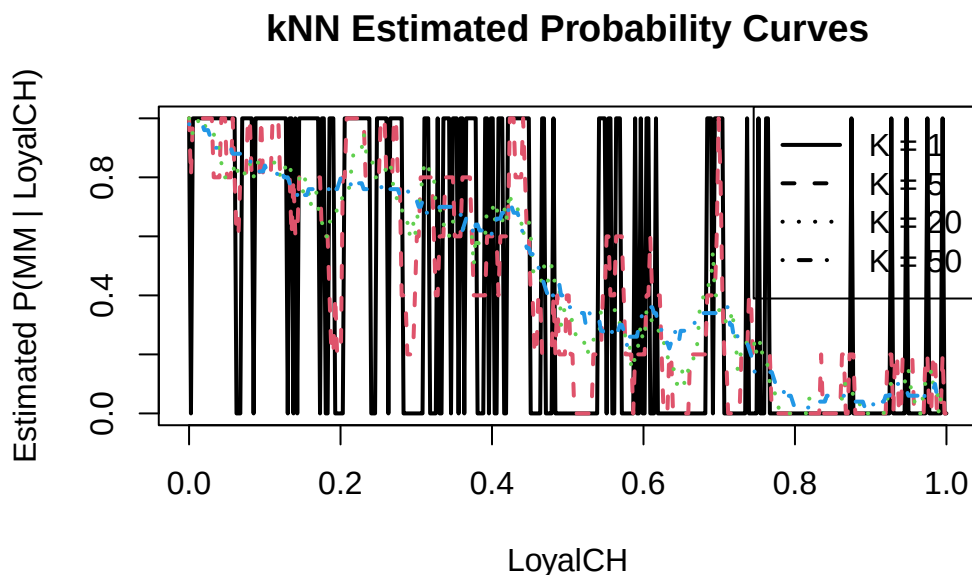
matplot(
```

```

loyal_knn_grid,
knn_grid_probs,
type = "l",
lty = 1:4,
lwd = 2,
xlab = "LoyalCH",
ylab = "Estimated P(MM | LoyalCH)",
main = "kNN Estimated Probability Curves"
)

legend(
  "topright",
  legend = paste("K =", K_values),
  lty = 1:4,
  lwd = 2
)

```



Q7. Which value of K gives the most flexible fit? Why?

$K = 1$ gives the most flexible fit. Each prediction depends only on the single nearest training observation, so the estimated probability can change abruptly as `LoyalCH` changes and is highly sensitive to individual observations.

Q8. Briefly describe what changes as K increases.

As K increases, each prediction averages over a larger neighborhood, so the estimated probability curve becomes smoother and less sensitive to individual observations. Thus, increasing K generally decreases variance but increases bias.

5. Choosing K

Use 5-fold cross-validation on the training data only.

```

set.seed(434634)

n_train <- nrow(OJ_train)

# Randomly assign approximately equal folds
fold_id <- sample(
  rep(1:5, length.out = n_train)
)

K_grid <- 1:50

cv_error <- numeric(length(K_grid))

for (K in K_grid) {

  fold_errors <- numeric(5)

  for (fold in 1:5) {

    cv_train <- fold_id != fold
    cv_valid <- fold_id == fold

    prob_valid <- knn_prob(
      matrix(OJ_train$LoyalCH[cv_train], ncol = 1),
      OJ_train$Y[cv_train],
      matrix(OJ_train$LoyalCH[cv_valid], ncol = 1),
      K
    )

    pred_valid <- ifelse(
      prob_valid >= 0.5,
      1,
      0
    )

    fold_errors[fold] <- mean(
      pred_valid != OJ_train$Y[cv_valid]
    )
  }

  cv_error[K] <- mean(fold_errors)
}

best_K_1d <- K_grid[which.min(cv_error)]

best_K_1d

```

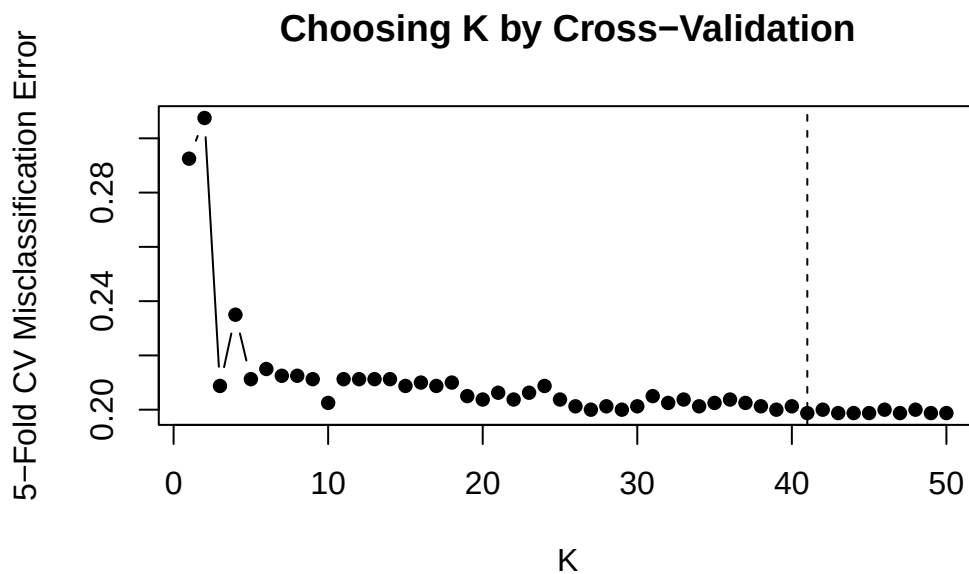
```
[1] 41
```

```
min(cv_error)
```

```
[1] 0.19875
```

```
plot(
  K_grid,
  cv_error,
  type = "b",
  pch = 16,
  xlab = "K",
  ylab = "5-Fold CV Misclassification Error",
  main = "Choosing K by Cross-Validation"
)

abline(
  v = best_K_1d,
  lty = 2
)
```



Evaluate the selected model once on the untouched test set.

```
prob_test_knn_1d <- knn_prob(
  matrix(OJ_train$LoyalCH, ncol = 1),
  OJ_train$Y,
  matrix(OJ_test$LoyalCH, ncol = 1),
  best_K_1d
)

pred_test_knn_1d <- ifelse(
  prob_test_knn_1d >= 0.5,
  1,
  0
)
```

```
)

knn_1d_test_error <- mean(
  pred_test_knn_1d != OJ_test$Y
)

knn_1d_test_error
```

```
[1] 0.1703704
```

Q9. Which value of K is selected by cross-validation?

Five-fold cross-validation selects $K = 41$, which has the smallest estimated CV misclassification error among $K = 1, \dots, 50$.

Q10. Why should the test set not be used to choose K ?

The test set should represent unseen data used only for final evaluation. If we use test performance to choose K , information from the test set influences model selection, so the resulting test error is optimistically biased and no longer provides an independent estimate of generalization performance.

6. Multivariable classification

Use the predictors

LoyalCH, PriceCH, PriceMM, DiscCH, DiscMM, SpecialCH, SpecialMM, and WeekofPurchase.

```
predictors <- c(
  "LoyalCH",
  "PriceCH",
  "PriceMM",
  "DiscCH",
  "DiscMM",
  "SpecialCH",
  "SpecialMM",
  "WeekofPurchase"
)

formula_multi <- as.formula(
  paste(
    "Y ~",
    paste(predictors, collapse = " + ")
  )
)
```

Logistic regression

```
logit_multi <- glm(
  formula_multi,
```

```

    data = OJ_train,
    family = binomial
)

prob_logit_multi <- predict(
  logit_multi,
  newdata = OJ_test,
  type = "response"
)

```

LDA

```

lda_multi <- lda(
  formula_multi,
  data = OJ_train
)

prob_lda_multi <- predict(
  lda_multi,
  newdata = OJ_test
)$posterior[, "1"]

```

QDA

```

qda_multi <- qda(
  formula_multi,
  data = OJ_train
)

prob_qda_multi <- predict(
  qda_multi,
  newdata = OJ_test
)$posterior[, "1"]

```

kNN

For kNN, standardize the continuous predictors using training-set means and standard deviations while leaving the binary indicators unchanged.

```

continuous_vars <- c(
  "LoyalCH",
  "PriceCH",
  "PriceMM",
  "DiscCH",
  "DiscMM",
  "WeekofPurchase"
)

```

```

binary_vars <- c(
  "SpecialCH",
  "SpecialMM"
)

X_train_multi <- OJ_train[, predictors]
X_test_multi  <- OJ_test[, predictors]

# Learn scaling parameters from training data only
train_means <- sapply(
  X_train_multi[, continuous_vars],
  mean
)

train_sds <- sapply(
  X_train_multi[, continuous_vars],
  sd
)

X_train_knn <- X_train_multi
X_test_knn  <- X_test_multi

X_train_knn[, continuous_vars] <- scale(
  X_train_multi[, continuous_vars],
  center = train_means,
  scale = train_sds
)

X_test_knn[, continuous_vars] <- scale(
  X_test_multi[, continuous_vars],
  center = train_means,
  scale = train_sds
)

# Binary variables remain unchanged
X_train_knn[, binary_vars] <- X_train_multi[, binary_vars]
X_test_knn[, binary_vars]  <- X_test_multi[, binary_vars]

X_train_knn <- as.matrix(X_train_knn)
X_test_knn  <- as.matrix(X_test_knn)

```

Choose K using 5-fold CV on the training data.

Importantly, the continuous predictors are standardized **inside each CV fold** using only that fold's training observations.

```
set.seed(434634)
```

```

fold_id_multi <- sample(
  rep(1:5, length.out = nrow(OJ_train))
)

K_grid_multi <- 1:50
cv_error_multi <- numeric(length(K_grid_multi))

for (K in K_grid_multi) {

  fold_errors <- numeric(5)

  for (fold in 1:5) {

    idx_train <- fold_id_multi != fold
    idx_valid <- fold_id_multi == fold

    fold_train <- OJ_train[idx_train, ]
    fold_valid <- OJ_train[idx_valid, ]

    X_fold_train <- fold_train[, predictors]
    X_fold_valid <- fold_valid[, predictors]

    # Standardization parameters from CV training fold only
    fold_means <- sapply(
      X_fold_train[, continuous_vars],
      mean
    )

    fold_sds <- sapply(
      X_fold_train[, continuous_vars],
      sd
    )

    X_fold_train_std <- X_fold_train
    X_fold_valid_std <- X_fold_valid

    X_fold_train_std[, continuous_vars] <- scale(
      X_fold_train[, continuous_vars],
      center = fold_means,
      scale = fold_sds
    )

    X_fold_valid_std[, continuous_vars] <- scale(
      X_fold_valid[, continuous_vars],
      center = fold_means,
      scale = fold_sds
    )
  }
}

```

```

# Leave binary variables as 0/1
X_fold_train_std[, binary_vars] <-
  X_fold_train[, binary_vars]

X_fold_valid_std[, binary_vars] <-
  X_fold_valid[, binary_vars]

prob_valid <- knn_prob(
  as.matrix(X_fold_train_std),
  fold_train$Y,
  as.matrix(X_fold_valid_std),
  K
)

pred_valid <- ifelse(
  prob_valid >= 0.5,
  1,
  0
)

fold_errors[fold] <- mean(
  pred_valid != fold_valid$Y
)
}

cv_error_multi[K] <- mean(fold_errors)
}

best_K_multi <-
  K_grid_multi[which.min(cv_error_multi)]

best_K_multi

```

```
[1] 18
```

```
min(cv_error_multi)
```

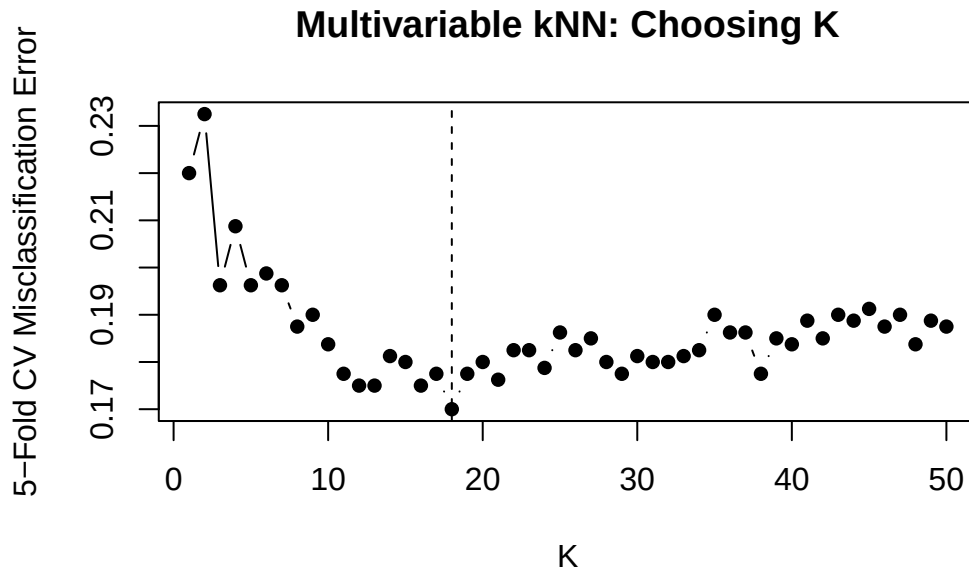
```
[1] 0.17
```

```

plot(
  K_grid_multi,
  cv_error_multi,
  type = "b",
  pch = 16,
  xlab = "K",
  ylab = "5-Fold CV Misclassification Error",
  main = "Multivariable kNN: Choosing K"
)

```

```
abline(
  v = best_K_multi,
  lty = 2
)
```



Fit the selected kNN classifier to the complete training set.

```
prob_knn_multi <- knn_prob(
  X_train_knn,
  OJ_train$Y,
  X_test_knn,
  best_K_multi
)
```

Now calculate test misclassification rate, log loss, and AUC for all four classifiers.

```
evaluate_classifier <- function(
  name,
  y_true,
  prob,
  threshold = 0.5
) {

  pred <- ifelse(
    prob >= threshold,
    1,
    0
  )

  data.frame(
    Method = name,
    Misclassification_Rate =
```

```

    mean(pred != y_true),
    Log_Loss =
      binary_log_loss(y_true, prob),
    AUC =
      auc_score(y_true, prob)
  )
}

results_multi <- rbind(
  evaluate_classifier(
    "Logistic Regression",
    OJ_test$Y,
    prob_logit_multi
  ),
  evaluate_classifier(
    "LDA",
    OJ_test$Y,
    prob_lda_multi
  ),
  evaluate_classifier(
    "QDA",
    OJ_test$Y,
    prob_qda_multi
  ),
  evaluate_classifier(
    paste0("kNN (K=", best_K_multi, ")"),
    OJ_test$Y,
    prob_knn_multi
  )
)

results_multi

```

	Method	Misclassification_Rate	Log_Loss	AUC
1	Logistic Regression	0.1703704	0.3954704	0.8987059
2	LDA	0.1629630	0.4002204	0.8990000
3	QDA	0.1962963	0.5677713	0.8697647
4	kNN (K=18)	0.1888889	0.5617521	0.8633529

Store the best-performing methods for the written responses.

```

best_error_method <-
  results_multi$Method[
    which.min(results_multi$Misclassification_Rate)
  ]

best_logloss_method <-
  results_multi$Method[

```

```

    which.min(results_multi$Log_Loss)
  ]

best_auc_method <-
  results_multi$Method[
    which.max(results_multi$AUC)
  ]

best_error_method

```

```
[1] "LDA"
```

```
best_logloss_method
```

```
[1] "Logistic Regression"
```

```
best_auc_method
```

```
[1] "LDA"
```

Q11. Which method has the smallest test misclassification rate?

The method with the smallest test misclassification rate is **LDA**, with a test error of 0.163. This means that, using a 0.5 classification threshold, it makes the fewest incorrect class predictions on the test set among the four methods considered.

Q12. Which method has the smallest test log loss? Is it necessarily the same method?

The method with the smallest test log loss is **Logistic Regression**, with log loss 0.3955. It is not the method with the smallest misclassification rate. These metrics need not select the same model because misclassification error considers only the final class label, whereas log loss also evaluates the quality and confidence of the estimated probabilities.

Q13. Briefly compare the four classifiers. Which would you choose for this problem, and why?

LDA and logistic regression perform best overall, while QDA and kNN perform noticeably worse. LDA has the lowest test misclassification rate (0.1630) and the highest AUC (0.8990), while logistic regression has the lowest log loss (0.3955). I would choose LDA because it performs best on two of the three evaluation metrics and remains relatively simple, although logistic regression is a close alternative when probability estimation is the main goal.

7. Takeaways

Q14. What is the difference between estimating $\widehat{P}(Y = 1 \mid X = x)$ and choosing a classification rule $\widehat{C}_D(x)$?

$\widehat{P}(Y = 1 \mid X = x)$ is an estimated conditional probability and describes the model's uncertainty about the positive class. A classification rule $\widehat{C}_D(x)$ converts probabilities into a discrete decision, typically by applying a threshold such as

$$\widehat{C}_D(x) = I\{\widehat{P}(Y = 1 \mid X = x) \geq 0.5\}.$$

Thus, probability estimation and decision-making are related but distinct steps.

Q15. Why can two classifiers have similar misclassification rates but different log loss or AUC?

Misclassification rate uses only the final class labels at a particular threshold, so it ignores how confident the predicted probabilities are. Log loss evaluates the probabilities themselves and strongly penalizes confident incorrect predictions, while AUC evaluates how well the model ranks positive observations above negative observations across all possible thresholds. Therefore, models with similar 0–1 classification accuracy can still differ substantially in probability quality or ranking ability.

Generative AI use

Generative AI was used to assist with debugging R code, along with checking the implementation of classification and cross-validation procedures.